

연락처 관리 웹 서비스 — 과제 목적 정의서 (2차 과제)

항목	내용
문서명	연락처 관리 웹 서비스(2차 과제) 과제 목적 정의서
문서 유형	과제 목적 (Objective / Background)
과제 구분	2차 과제 — FastAPI + DB 연락처 프로그램
선행 과제	1차 과제 — 콘솔 연락처 관리 프로그램 (완료를 전제로 함)
버전	v1.0
기술 스택	Python + FastAPI + PostgreSQL 16 + 세션 로그인 + HTML/JavaScript
대상	교육과정 수강생 (파이썬 기초 완료자, 웹 개발 처음)
상태	확정(Baseline)

과제 체계 (1차 → 2차)

이 교육과정의 실습 과제는 같은 프로그램을 두 번, 다른 도구로 만드는 2단계 구성입니다.

구분	1차 과제 (완료)	2차 과제 (본 문서)
과제명	콘솔 연락처 관리 프로그램	FastAPI + DB 연락처 프로그램
실행 환경	터미널(콘솔)	웹 브라우저
데이터 보관	딕셔너리 + pickle 파일	PostgreSQL 16 (테이블 3개)
사용자	1명 (실행한 사람)	여러 명 (회원가입 + 로그인, 데이터 격리)
종류(카테고리)	가족/친구/기타 3개 고정	사용자가 직접 추가/수정/삭제
핵심 학습	CRUD · 딕셔너리 · 파일 저장 · 예외 처리	REST API · DB(FK) · 인증 · 화면 연동

왜 같은 프로그램을 두 번 만드나요? 기능(연락처 추가/조회/수정/삭제)이 같기 때문에, 2차 과제에서는 "무엇을 만들지" 고민할 필요 없이 "실무 도구로 어떻게 만드는지"에만 집중할 수 있습니다. 1차에서 손으로 짰던 것들이 2차에서 어떤 실무 도구로 바뀌는지 비교하며 배우는 것이 이 과제 체계의 핵심입니다.

검증된 기술 스택 버전 (실제 설치 확인)

아래 버전은 문서 작성일에 PyPI에서 실제로 함께 설치하여 버전 충돌이 없음을 확인한 조합입니다.

구성요소	검증 버전	역할
Python	3.12.3 (검증 실행) / 3.14.x (최신 안정 라인)	실행 환경 (FastAPI는 3.10+ 요구)
FastAPI	0.139.0	웹 프레임워크 (API 서버)
Pydantic	2.13.4	입력 데이터 자동 검증
SQLAlchemy	2.0.51	ORM (파이썬 ↔ DB 연결)
psycopg	3.3.4	PostgreSQL 드라이버
pwdlib[argon2]	0.3.0	비밀번호 해싱 — Argon2 (FastAPI 공식 문서 권장)
Uvicorn	0.50.0	ASGI 웹 서버
PostgreSQL	16 (Docker)	데이터베이스

인증 방식 선정 근거: 인증은 **세션 쿠키 로그인**을 채택합니다. ① 추가 라이브러리가 0개입니다 — 세션 ID 생성은 파이썬 표준 라이브러리 `secrets` 하나면 됩니다. ② 쿠키는 **브라우저가 자동으로 보관·첨부**하므로, 화면(JavaScript) 쪽에 토큰 저장/첨부 코드를 짤 필요가 없습니다. ③ Django·Flask 등 전통 웹 프레임워크의 기본 로그인 방식이 바로 세션이며, 지금도 실무에서 널리 쓰이는 표준 방식입니다. (모바일 앱·마이크로서비스에서 쓰는 JWT 방식은 심화 과정 주제로 미룹니다.) 본 문서 작성일에 **가입→로그인→쿠키 자동 첨부→사용자별 데이터 격리→로그아웃** 전체 흐름을 실제 FastAPI 앱으로 실행하여 확인했습니다. 비밀번호는 어떤 방식이든 원문 저장이 금지이므로, 해싱(pwdlib·Argon2)은 그대로 유지합니다(함수 2개 호출이 전부입니다).

이 문서를 읽는 분께(안내) 이 문서는 "왜 이 과제를 하는가"를 설명합니다. 코드를 어떻게 짜는지(=How)는 뒤의 **구현요구사항, 화면정의서, 기능정의서, PRD, TRD** 문서에서 다룹니다. 이 문서는 "무엇을 배우고, 그것이 실제 회사에서 어떻게 쓰이는가(=Why)"에만 집중합니다.

1. 과제 개요

1차 과제에서 여러분은 **콘솔 연락처 관리 프로그램**(딕셔너리 + pickle 파일)을 완성했습니다. 2차 과제는 그 기능을 실무에서 실제로 쓰는 방식인 **웹 서비스**로 다시 만들고, 여기에 실무 서비스라면 반드시 있는 두 가지를 추가합니다.

- 로그인 (멀티 유저)** — 여러 사용자가 각자 회원가입/로그인하고, **자기 회원 목록만** 관리합니다. (카카오톡에서 내 친구 목록은 나만 보이는 것과 같은 구조)
- 종류(카테고리) 관리** — 1차 과제에서는 "가족/친구/기타" 3개로 고정이었지만, 이번에는 사용자가 카테고리를 **직접 추가/수정/삭제**할 수 있습니다. (예: "동호회", "거래처" 추가)

이 과제의 목적은 다음 한 문장으로 요약됩니다.

"콘솔에서 직접 만들었던 CRUD·검증·저장의 원리를 실무 표준 도구(REST API + 데이터베이스 + 웹 화면)로 옮기고, 로그인(인증)과 데이터 격리, 테이블 간 관계(FK)까지 갖춘 '진짜 서비스의 최소 완전체'를 만든다."

핵심 포인트는 이것입니다. **CRUD 골격은 이미 아는 것 그대로이고, 거기에 실무 서비스의 필수 요소 2개(누가 쓰는가 = 인증, 데이터끼리의 관계 = FK)가 얹힙니다.** 이 두 가지가 더해지는 순간, 과제가 "연습 프로그램"에서 "포트폴리오에 넣을 수 있는 서비스"로 바뀝니다.

용어 구분 (혼동 주의 — 문서 세트 전체 공통) 멀티 유저가 되면서 "가입해서 로그인하는 사람"과 "저장·관리되는 데이터"를 구분할 필요가 생깁니다. 본 문서 세트는 다음과 같이 구분합니다. (1차 과제에서 "회원"이라 부르던 데이터가 2차 과제의 "연락처"입니다.)

용어	의미	예	테이블
사용자(User)	이 서비스에 로그인하는 사람	happyday 계정	users
연락처(Contact)	사용자가 관리하는 연락처 데이터	윤아, 은혁	contacts

즉, "사용자 happyday가 로그인해서 자신의 연락처 윤아를 등록한다"라고 읽으면 됩니다. 휴대폰 주소록 앱에서 "내 계정"과 "저장된 연락처"의 관계와 같습니다.

2. 1차 과제(콘솔) → 2차 과제(웹): 무엇이, 왜 바뀌는가

두 버전을 나란히 놓고 보면 이번 과제에서 배울 것이 한눈에 보입니다.

역할	1차 과제 (콘솔)	2차 과제 (웹)	실무에서 부르는 이름
사용자 화면	터미널 메뉴 (<code>input()</code>)	브라우저 HTML 화면	프론트엔드 (Frontend)
기능 요청	메뉴 번호 입력 (1~5)	HTTP 요청 (GET/POST/PATCH/DELETE)	REST API
데이터 검증	<code>if</code> + 정규식 직접 작성	Pydantic 스키마 (자동 검증)	서버 입력 검증
데이터 처리	딕셔너리 조작	SQLAlchemy ORM → SQL	백엔드 로직
데이터 저장	pickle 바이너리 파일	PostgreSQL 테이블	데이터베이스(RDB)
데이터 구조	<code>dict</code> 1개	<code>users</code> · <code>categories</code> · <code>contacts</code> 3개 테이블 (관계 연결)	데이터 모델링 / FK
저장 시점	종료 시 1회 (<code>pickle.dump</code>)	요청 처리 즉시 (트랜잭션 커밋)	영속화(Persistence)
오류 표시	<code>print("잘못된 입력입니다")</code>	HTTP 상태 코드 (401, 404, 409, 422)	에러 응답 규약
사용자 개념	없음 (프로그램 실행자 1명)	회원가입 + 세션 로그인, 사용자별 데이터 격리	인증 (Authentication)
종류(가족/친구/기타)	코드에 고정된 3개 값	사용자가 직접 추가/수정/삭제하는 카테고리	참조 테이블 관리

비유로 이해하기 — "포장마차에서 정식 식당으로" 1차 과제(콘솔)는 혼자 운영하는 포장마차입니다. 주문(입력)도, 조리(처리)도, 장부 기록(pickle 저장)도 한 사람이 한 자리에서 다 합니다. 2차 과제(웹)는 **역할이 분리된 정식 식당**입니다. 손님이 보는 **메뉴판·홀**(HTML 화면), 주문을 받아 주방에 전달하는 **주문 창구**(REST API), 요리하는 **주방**(FastAPI 로직), 재료와 장부를 보관하는 **창고**(PostgreSQL)가 나뉩니다. 파는 음식(기능)은 같지만, **규모를 키울 수 있는 구조**가 됩니다. 실무의 모든 서비스가 이 구조입니다.

3. 무엇을 배우는가 (핵심 학습 목표 7가지)

3-1. CRUD → REST API : "메뉴 번호"가 "HTTP 메서드"가 된다

1차 과제의 메뉴 1~5번은, 웹에서는 **주소(URL) + HTTP 메서드** 조합으로 표현됩니다. 그리고 이번 과제에서는 같은 CRUD 패턴이 **연락처(contacts)**와 **카테고리(categories)** 두 자원에 반복되고, 그 앞에 **인증 관문(auth)** 이 붙습니다.

1차 과제 메뉴	동작	2차 과제 (REST API)	실무에서 같은 일
(없음 — 신규)	가입/로그인/로그아웃	POST /auth/signup · /auth/login · /auth/logout	앱 첫 화면의 회원가입/로그인
1. 연락처 추가	Create	POST /contacts	연락처 앱에서 새 연락처 저장
2. 연락처 목록 보기	Read	GET /contacts	내 연락처 목록 열기 (내 것만 보임)
3. 연락처 정보 수정	Update	PATCH /contacts/{id}	연락처 정보 변경 저장
4. 연락처 삭제	Delete	DELETE /contacts/{id}	연락처 삭제
(없음 — 신규)	카테고리 CRUD	POST/GET/PATCH/DELETE /categories	연락처 앱의 "그룹 관리" 기능
5. 종료(저장)	-	(불필요 — DB가 즉시 저장)	-

주목할 점: 카테고리 API를 표에 추가해도 새로 배울 것이 거의 없습니다. **POST/GET/PATCH/DELETE** 패턴이 자원 이름만 바뀌 **그대로 반복**되기 때문입니다. "CRUD 패턴 하나를 익히면 자원이 몇 개 되어도 같은 방법으로 만든다" — 이것이 REST API 설계의 힘이고, 이번 과제에서 직접 체감하게 됩니다.

비유로 이해하기 콘솔 메뉴는 "식당에 한 개뿐인 주문 창구에서 번호표로 주문"하는 방식입니다. REST API는 "용건 별 전용 창구"입니다. 등록 창구(POST), 조회 창구(GET), 변경 창구(PATCH), 해지 창구(DELETE)가 따로 있어서, 창구 이름만 봐도 무슨 일을 하는지 알 수 있습니다.

실무 연결 여러분이 쓰는 모든 앱(배달앱, 은행앱, SNS)의 버튼 하나하나가 전부 이런 API 호출입니다. "연락처 저장 버튼 = **POST /contacts**"라는 감각이 백엔드 개발의 출발점입니다.

3-2. 딕셔너리 → 데이터베이스 테이블 : "수첩"이 "장부 시스템"이 된다

1차 과제에서 연락처 1건은 딕셔너리 1개였습니다. 2차 과제에서는 **테이블의 행(row) 1개**가 됩니다.

1차 과제(콘솔): 딕셔너리

```
member = {"name": "윤아", "phone": "01012345678", "addr": "서울시", "type": "친구"}
```

-- 2차 과제(웹): PostgreSQL 테이블의 행

```
SELECT * FROM contacts;
```

```
id | user_id | category_id | name | phone | addr
```

```
---+-----+-----+-----+-----+-----+
1 |      1 |          2 | 윤아 | 01012345678 | 서울시
```

↑ user_id = 이 연락처의 주인 (로그인 사용자)

category_id = categories 테이블의 "친구" 행을 가리킴

딕셔너리의 **key**가 테이블의 **열(column)** 이 되고, 연락처 한 건이 **행(row)** 이 됩니다. 구조가 1:1로 대응하기 때문에, 딕셔너리를 이해했다면 테이블도 이미 절반은 이해한 것입니다.

그런데 위 테이블을 잘 보면 1차 과제와 다른 점이 있습니다. 1차 과제의 "종류" 항목이 글자가 아니라 **다른 테이블을 가리키는 번호(category_id)**로 바뀌었습니다. 이것이 이번 과제의 핵심 개념인 **외래키(Foreign Key, FK)**입니다. (자세한 내용은 3-7)

실무 연결 실무 데이터의 표준 보관소는 파일이 아니라 **관계형 데이터베이스(RDB)**입니다. 그리고 파이썬에서 SQL을 직접 쓰는 대신 **ORM(SQLAlchemy)**으로 "파이썬 클래스 ↔ 테이블"을 연결하는 것이 현대 백엔드의 표준 방식입니다. 이번 과제에서 그 표준을 그대로 사용합니다.

3-3. pickle → DB 영속화 : "종료 시 저장"이 "즉시 저장"이 된다

1차 과제의 가장 큰 약점을 기억하나요? **종료(메뉴 5) 전에 프로그램이 강제 종료되면 그날 작업이 전부 사라졌습니다.** 데이터베이스는 이 문제를 구조적으로 해결합니다.

구분	pickle 파일	PostgreSQL
저장 시점	종료 시 1회 (직접 호출)	요청 처리마다 즉시 (커밋)
비정상 종료 시	마지막 저장 이후 데이터 소실	커밋된 데이터는 안전
동시 사용	불가 (파일 1개를 통째로 덮어씀)	여러 사용자 동시 처리 가능
일부만 수정	전체를 다시 저장	해당 행만 UPDATE

비유로 이해하기 pickle은 "하루 장사를 마치고 퇴근할 때 장부를 한 번에 정리"하는 방식입니다. 퇴근 전에 불이 나면 그날 기록이 다 사라집니다. DB는 "거래가 발생할 때마다 즉시 장부에 기록하고 금고에 넣는" 방식입니다. 이 "거래 단위의 확실한 기록"을 실무에서는 **트랜잭션(Transaction)**이라고 부릅니다.

3-4. 수동 유효성 검사 → Pydantic 자동 검증 : "검사 코드"가 "검사 규칙 선언"이 된다

1차 과제에서는 `validate_name`, `validate_phone` 함수를 직접 만들고, 실패하면 재입력 루프를 직접 돌렸습니다. FastAPI에서는 규칙만 선언하면 검사·오류응답이 자동입니다.

```
# 1차 과제(콘솔): 검사 로직을 직접 작성 (약 20줄의 함수 + 재입력 루프)
def validate_name(name):
    return 1 <= len(name) <= 5

# 2차 과제(웹): 규칙을 "선언"만 하면 FastAPI가 검사·422 응답까지 자동 처리
class MemberCreate(BaseModel):
    name: str = Field(min_length=1, max_length=5)
    phone: str = Field(pattern=r"^010\d{8}$")
```

실무 연결 "검사 로직을 매번 손으로 짜지 않고, 규칙을 선언하면 프레임워크가 실행한다" — 이것이 실무 프레임워크의 핵심 철학입니다. 규칙이 코드 한곳에 모여 있으니 실수도 줄고, API 문서(</docs>)에 규칙이 자동으로 표시되어 협업도 쉬워집니다. 다만 "동명이인 선택", "전화번호 중복 금지"처럼 DB를 봐야 하는 검증은 여전히 직접 짜야 한다는 것도 함께 배웁니다.

3-5. 프론트엔드 ↔ 백엔드 연동 : "한 프로그램"이 "두 프로그램의 대화"가 된다

1차 과제(콘솔)는 화면과 로직이 한 파일 안에 있었습니다. 2차 과제에서는 브라우저(프론트엔드)와 서버(백엔드)가 HTTP + JSON으로 대화하는 별개의 프로그램이 됩니다.

```
[브라우저] "연락처 추가해줘" — POST /contacts + JSON —▶ [FastAPI 서버]
[브라우저] ◀— 201 Created + 등록된 연락처 JSON — [FastAPI 서버]
```

여러분이 이미 배운 JavaScript의 `fetch()`와 `Promise/async-await`가 바로 이 대화를 담당합니다. 이번 과제에서 앞서 배운 JavaScript(DOM/이벤트, 비동기)와 파이썬(FastAPI)이 처음으로 하나의 시스템 안에서 만나게 됩니다.

실무 연결 실무 개발팀이 "프론트엔드 개발자 / 백엔드 개발자"로 나뉘는 이유가 바로 이 구조 때문입니다. 두 역할이 API 명세(어떤 주소로, 어떤 JSON을 주고받을지)를 계약서 삼아 협업합니다. 이번 과제에서 여러분은 혼자서 양쪽을 다 만들며 그 "계약"이 무엇인지 체험합니다.

3-6. 로그인(세션 쿠키)과 데이터 격리 : "내 것만 보인다"를 만드는 방법 (신규)

콘솔 프로그램에는 "사용자"라는 개념이 없었습니다. 실행한 사람이 곧 주인이었으니까요. 하지만 웹 서버는 여러 사람의 요청을 동시에 받습니다. 그래서 두 가지 질문에 매 요청마다 답해야 합니다.

1. "지금 요청한 사람이 누구인가?" → 인증(Authentication)
2. "그 사람의 데이터만 보여주고 있는가?" → 데이터 격리(Data Isolation)

이번 과제에서는 웹의 가장 전통적이고 단순한 방식인 세션(Session) 로그인을 사용합니다. 원리는 딱 3단계이고, 준비물은 테이블 1개(`sessions`)와 파이썬 표준 기능(`secrets`) 뿐입니다.

- ① 로그인: 아이디/비밀번호 제출 → 서버가 확인 후, 무작위 "세션 번호"를 만들어
 - ⓐ sessions 테이블(장부)에 "이 번호 = happyday" 라고 기록하고
 - ⓑ 그 번호를 쿠키(cookie)에 담아 브라우저에 돌려준다
- ② 보관: 브라우저가 쿠키를 알아서 보관한다 (우리가 짤 코드 없음!)
- ③ 사용: 이후 모든 요청에 브라우저가 쿠키를 알아서 첨부한다
→ 서버는 쿠키 속 번호를 장부에서 찾아 "아, happyday구나" 하고 알아낸다

비유로 이해하기 — "헬스장 출입 카드와 프런트 장부" 헬스장에 처음 등록(회원가입)하면 신분 확인 후 **출입 카드**(세션 쿠키)를 줍니다. 프런트 **장부**(sessions 테이블)에는 "카드 37번 = 홍길동"이라고 적어 둡니다. 이후 운동하러 올 때 마다(API 요청마다) 신분증을 다시 보여줄 필요 없이 **카드만 찍으면**, 프런트가 장부를 보고 누군지 압니다. 회원 탈퇴나 카드 반납(로그아웃)을 하면 **장부에서 그 줄을 지우기만 하면** 카드는 즉시 무효가 됩니다.

이 방식이 처음 배우는 사람에게 좋은 이유는 명확합니다.

학생 입장에서의 장점	설명
새 라이브러리가 없다	세션 번호 생성은 표준 라이브러리 <code>secrets.token_hex()</code> 한 줄
화면 코드가 단순하다	쿠키는 브라우저가 자동 보관·자동 첨부 → JavaScript에 토큰 저장/첨부 코드 불필요
눈으로 확인 가능하다	로그인 상태가 sessions 테이블의 행 으로 보임 → PostgreSQL에서 SELECT 로 직접 조회 가능
로그아웃이 직관적이다	장부(테이블)에서 행 하나 삭제 = 즉시 로그아웃

비밀번호 보관에는 방식과 무관한 절대 규칙이 하나 있습니다. **비밀번호는 절대 원문 그대로 저장하지 않는다**. 되돌릴 수 없는 형태(해시)로 변환해 저장하며, 이번 과제는 FastAPI 공식 문서가 권장하는 **Argon2 해시(pwdlib)** 를 사용합니다. 어렵지 않습니다 — 저장할 때 `hash()`, 로그인 때 `verify()`, 함수 딱 2개입니다.

실무 연결 세션 로그인은 "옛날 방식"이 아닙니다. Django, Flask 같은 전통 웹 프레임워크의 기본 로그인인 정확히 이 구조이고, 지금도 수많은 서비스가 세션으로 동작합니다. (모바일 앱·서버 간 통신에서 많이 쓰는 JWT 토큰 방식은 이 세션 개념을 이해한 뒤 배우는 심화 주제입니다.) 그리고 인증보다 더 중요한 것이 **데이터 격리**입니다 — 실무에서 가장 치명적인 사고(개인정보 유출)는 "로그인은 만들었는데, 조회/수정/삭제 어딘가에서 `user_id` 필터를 빠뜨린" 경우에 터집니다. 이번 과제의 채점도 이 부분을 가장 무겁게 봅니다. (B가 A의 연락처 id를 알아내 `DELETE /contacts/{id}` 를 호출해도 404가 나와야 합니다.)

3-7. 카테고리 분리 : "값"이 "관계"가 된다 — 외래키(FK) 입문 (신규)

1차 과제의 종류(가족/친구/기타)는 코드에 박힌 고정 값이었습니다. 사용자가 "동호회"를 추가하고 싶어도 방법이 없었죠. 이번에는 카테고리를 **별도 테이블로 분리**해서 사용자가 직접 관리하게 만듭니다.

- [1차 과제] `member["type"] = "친구"`  글자를 직접 저장 (오타·표기 불일치 위험)
- [2차 과제] categories 테이블: id=1 가족 / id=2 친구 / id=3 기타 / id=4 동호회(사용자 추가)
 `contacts.category_id = 2`  "친구" 행의 번호를 가리킴 (외래키)

이렇게 바꾸면 세 가지가 좋아집니다.

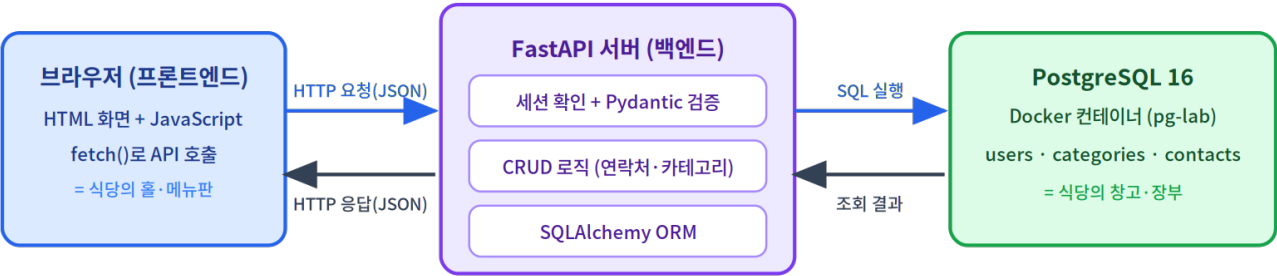
효과	설명	1차 과제에서의 문제
자유로운 관리	카테고리를 추가/수정/삭제 가능	"가족/친구/기타" 3개 고정
이름 변경이 한 번에	"친구"→"베프"로 바꾸면 소속 연락처 전체에 즉시 반영	연락처마다 저장된 글자를 전부 수정해야 함
무결성 보장	없는 카테고리 번호는 DB가 거부 (FK 제약)	"친구들", "칭구" 같은 오타도 그냥 저장됨

동시에 새로운 고민도 생깁니다 — "연락처가 소속된 카테고리를 삭제하려 하면?" 본 과제는 실무에서 가장 안전한 기본값인 "사용 중이면 삭제 거부(409 응답)" 를 채택합니다. (1차 과제의 "동명이인 처리"처럼, 이번 과제의 대표 옛지 케이스입니다.)

실무 연결 "반복되는 값을 별도 테이블로 분리하고 번호로 참조한다" — 이것이 관계형 DB 설계의 출발점(정규화)입니다. 쇼핑몰의 상품↔카테고리, 게시판의 글↔작성자가 전부 이 구조입니다. PostgreSQL 커리큘럼에서 배운 JOIN이 여기서 실제로 사용됩니다(연락처 목록에 카테고리 이름을 붙여서 조회).

4. 전체 아키텍처 (Architecture)

이번 과제에서 만드는 시스템의 전체 구조입니다. 상자 하나하나가 "역할이 분리된 부품"입니다.

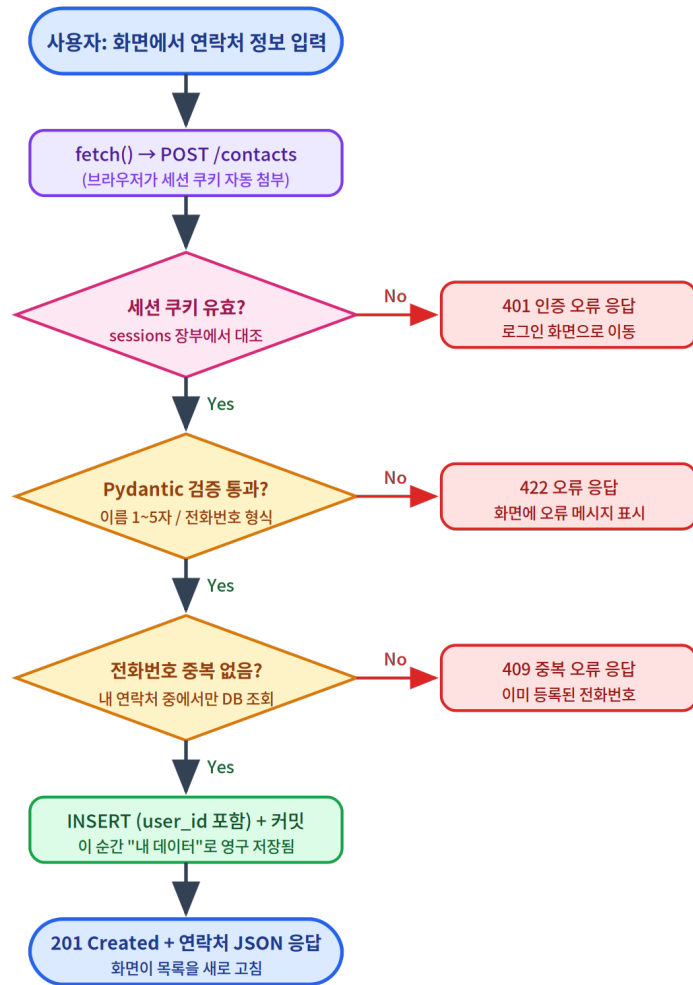


브라우저는 서버와만, 서버는 DB와만 대화한다 — 역할 분리가 웹 아키텍처의 핵심

계층	담당	1차 과제에서의 대응물
프론트엔드 (브라우저)	화면 표시, 입력 수집, API 호출	<code>print()</code> 메뉴 + <code>input()</code>
백엔드 (FastAPI)	검증, CRUD 로직, 응답 생성	<code>validate_*</code> + <code>add_member</code> 등 함수들
데이터베이스 (PostgreSQL)	데이터 영구 보관, 무결성 보장	딕셔너리 + pickle 파일

5. 전체 워크플로우 (Overall Workflow)

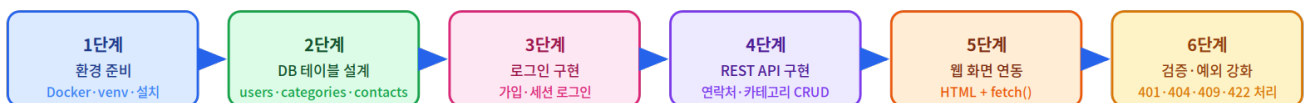
"연락처 추가" 요청 1건이 시스템을 통과하는 전체 흐름입니다. 1차 과제의 **입력 → 검증 → 처리 → 저장 → 조회** 골격이 그대로 유지되면서, 각 단계의 담당자만 바뀐 것에 주목하세요.



1차 과제와 비교하면: "재입력 루프"가 "오류 응답 + 화면 표시"로, "딕셔너리에 추가"가 "INSERT + 커밋"으로, "메뉴 복귀"가 "화면 새로 고침"으로 바뀌었을 뿐, **흐름의 뼈대는 동일합니다.** 유일하게 완전히 새로운 단계는 맨 앞의 **세션 확인(401)**입니다 — 콘솔에는 "남"이 없었지만, 웹에는 있기 때문입니다.

6. 단계별 학습 워크플로우 (Step Workflow)

이 과제는 아래 5단계 순서로 진행합니다. 각 단계가 끝날 때마다 **동작을 직접 확인**하고 다음으로 넘어갑니다.



각 단계 완료 시 반드시 실행하여 확인 후 다음 단계로 진행 (1차 과제와 동일한 원칙)

단계	하는 일	완료 확인 방법
1	Docker로 PostgreSQL 16 실행, 가상환경 + FastAPI 설치	<code>/docs</code> 페이지가 브라우저에 열림
2	<code>users</code> · <code>categories</code> · <code>contacts</code> 테이블 설계 (SQLAlchemy 모델, FK 연결)	DB에 테이블 3개 생성 확인
3	회원가입·로그인·로그아웃 API + 세션 쿠키 발급/확인 구현	<code>/docs</code> 에서 로그인 실행 후 보호 API 호출 성공 (쿠키는 자동 첨부됨)
4	연락처·카테고리 CRUD API 구현 (모든 API에 사용자 격리 적용)	계정 2개로 테스트: 서로의 데이터가 안 보임
5	HTML 화면 + fetch 연동 (로그인 화면 → 관리 화면)	브라우저 화면에서 로그인부터 관리까지 가능
6	미인증(401)·없는 연락처(404)·중복/사용 중 카테고리(409)·형식 오류(422) 처리	테스트 시나리오 전체 통과

7. 이 과제의 실무적 위치 (왜 또 연락처 관리인가)

1차 과제에서 "연락처 관리는 모든 백엔드의 원형"이라고 했습니다. 2차 과제는 그 원형을 **실무 도구 그대로** 구현하는 것입니다. 이번에는 화살표의 왼쪽이 아니라 **오른쪽을 직접 만듭니다**.

[1차 과제]	[2차 과제]	[실무 시스템]
연락처 디렉터리	→ PostgreSQL 테이블 3개	= 관계형 데이터 모델 (FK)
pickle 파일	→ DB 트랜잭션	= 관계형 DB 영속화
콘솔 메뉴 입력	→ REST API 엔드포인트	= REST API 엔드포인트
<code>validate_*</code> 함수	→ Pydantic 스키마	= 서버 입력 검증
<code>print()</code> 출력	→ HTML + fetch 화면	= 프론트엔드
(없음)	→ 회원가입 + 세션 로그인	= 인증/인가 (Auth)
종류 고정 3개	→ 카테고리 CRUD	= 참조(마스터) 데이터 관리

즉, 이번 과제를 완성하면 **"로그인이 있는 멀티 유저 CRUD 서비스"** — 실무 신입 개발자가 첫 업무로 받는 과제이자 취업 포트폴리오의 최소 완전체 — 를 처음부터 끝까지 만들어 본 것이 됩니다. 실제 서비스(연락처 앱, 메모 앱, 가계부 앱)와 구조가 동일합니다.

8. 학습 후 도달 목표 (역량 기준)

이 과제를 완료한 수강생은 다음을 혼자서 할 수 있어야 합니다.

1. Docker로 PostgreSQL을 실행하고, 파이썬(SQLAlchemy)에서 접속할 수 있다.
2. 요구사항의 데이터 항목을 **테이블 3개(users/categories/contacts)**로 설계하고, 기본키(id)·고유 제약(사용자별 전화번호 UNIQUE)·외래키(FK)의 필요성을 설명할 수 있다.
3. 비밀번호를 해시(Argon2)로 저장하고, 로그인 시 세션 번호를 발급해 쿠키로 전달하고, 이후 요청에서 세션 테이블과 대조하는 흐름을 구현할 수 있다.
4. 모든 조회/수정/삭제에 `user_id` 필터를 적용해 사용자 간 데이터를 격리할 수 있다. (남의 데이터 id로 접근 시 404)
5. CRUD 각각을 HTTP 메서드 + URL + 상태 코드로 설계하고, 같은 패턴을 연락처와 카테고리 두 자원에 적용할 수 있다.

- Pydantic으로 입력 규칙(길이, 정규식)을 선언하고 422 자동 응답을 확인하며, "없는 연락처(404)", "중복 전화번호 (409)", "사용 중인 카테고리 삭제(409)" 같은 **DB를 봐야 아는 예외**를 직접 처리할 수 있다.
- HTML + JavaScript `fetch()` 로 로그인부터 CRUD까지 화면에서 동작하게 연동할 수 있다. (세션 쿠키는 브라우저가 자동 처리함을 이해)
- Swagger UI(`/docs`)와 테스트 코드로 **자신의 API를 스스로 검증**할 수 있다. (한 번 로그인하면 쿠키가 자동 첨부되어 보호 API도 바로 테스트 가능)

9. 평가 관점 (무엇을 잘하면 잘한 것인가)

평가 영역	잘된 예	흔한 실수
인증 (최중요)	로그인 없이 접근 시 401, 비밀번호는 해시 저장	비밀번호 원문 저장, 보호 안 된 API 존재
데이터 격리 (최중요)	남의 데이터 id로 접근 시 404	조회는 격리했지만 수정/삭제에서 <code>user_id</code> 필터 누락
기능 완성도	인증 + 연락처/카테고리 CRUD + 화면이 모두 동작	카테고리 수정/삭제 기능 누락
데이터 영속화	서버 재시작 후에도 데이터 유지	커밋 누락으로 데이터가 저장 안 됨
관계(FK) 처리	사용 중 카테고리 삭제 시 409 안내	FK 오류가 그대로 500으로 노출됨
유효성 검사	잘못된 전화번호에 422 응답	검증 없이 그냥 저장
API 설계	메서드·상태 코드가 의미에 맞음	모든 요청을 GET/POST로만 처리
화면 연동	처리 결과가 화면에 즉시 반영	등록 후 새로고침해야 보임
코드 구조	계층 분리 (라우터/CRUD/모델/스키마)	모든 코드가 <code>main.py</code> 한 덩어리

10. 후속 문서 안내

순서	문서 (2차 과제 문서 세트)	다루는 내용
00	과제목적 정의서 (본 문서)	왜 이 과제를 하는가, 무엇을 배우는가
01	구현요구사항 정의서	데이터 모델(3테이블), 인증·연락처·카테고리 API 기능 요구, 유효성·예외 규칙 명세
02	화면 정의서	웹 화면(로그인/관리 화면) 레이아웃, 화면 전이, Swagger UI 활용
03	기능 정의서	엔드포인트·함수 단위 명세, 모듈 호출 관계
04	PRD (제품 요구사항)	사용자 관점 요구사항, 유스케이스, 인수 기준
05	TRD (기술 요구사항)	아키텍처, DB 설계(DDL·FK), 인증 설계(세션), 모듈 설계, 기술 스택·버전

문서 읽는 순서 권장: 과제목적 → 구현요구사항 → 화면정의서 → 기능정의서 → PRD → TRD (1차 과제 문서 세트와 동일한 순서)